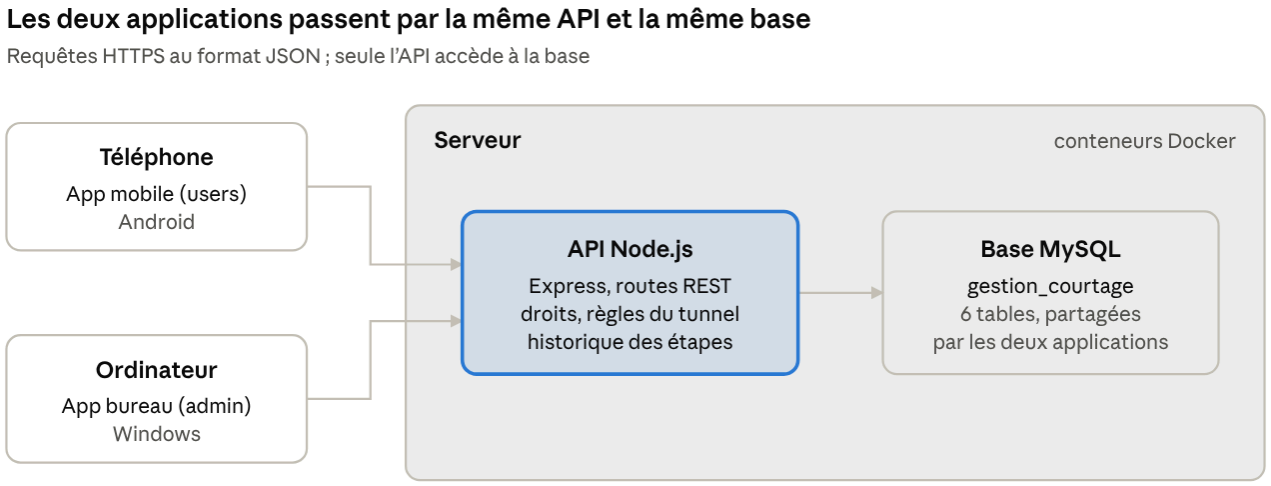
Application de gestion de cabinet de courtage — Cahier technique

Oct 21, 2026 · @Dorian

Version 1 : tunnel de vente. Ce document décrit la manière dont l'application est construite : architecture, technologies retenues et justifications, base de données, sécurité et démonstration.

Architecture générale



architecture · 2 applications, 1 API, 1 base MySQL

L'application mobile (users) et l'application bureau (admin) envoient leurs demandes à la même API. C'est l'API qui vérifie les droits, applique les règles du tunnel de vente et lit ou écrit dans la base ; les deux applications voient donc toujours les mêmes données.

Choix techniques et justifications

Besoin	Choix	Justification
Applications mobile et bureau	Flutter (Dart), deux projets	Même langage pour les deux applications ; technologie déjà utilisée sur un projet précédent.
Serveur (API)	Node.js, framework Express	Technologie maîtrisée. Express est minimaliste : chaque route et chaque vérification sont écrites explicitement, ce qui facilite l'apprentissage.

Besoin	Choix	Justification
Base de données	MySQL	Données fortement liées (un client a des affaires, une affaire porte sur un produit) : les clés étrangères garantissent la cohérence.
Exécution du serveur	Docker et Docker Compose	La base et l'API démarrent avec une seule commande, de la même façon sur n'importe quel poste.
Versionnage	Git et GitHub	Historique du code et sauvegarde du travail.

Applications Flutter

Deux applications distinctes : l'application mobile (Android) pour les users et l'application bureau (Windows) pour l'admin.

Rôle des applications

- Afficher les écrans, contrôler la saisie (champs obligatoires, formats) et envoyer les requêtes à l'API.
- Aucune règle déterminante côté application : droits et calculs sont faits par l'API.
- Seul le jeton de connexion est conservé sur l'appareil, dans le stockage sécurisé du système.

Écrans de la version 1

Application mobile	Application bureau
Connexion	Connexion
Liste et fiche des clients	Gestion des users
Liste des affaires par étape	Catalogue (compagnies, produits)
Détail d'une affaire (changement d'étape, devis, signature, perte)	Toutes les affaires, avec filtres et historique
Mes chiffres	Tableau de bord

Organisation du code : écrans, modèles (objets métier), services (appels à l'API). La gestion d'état, c'est-à-dire la mise à jour automatique des écrans quand les données changent, utilise **Provider**, la solution présentée par la documentation officielle de Flutter.

API Node.js

L'API est le seul composant qui accède à la base de données. Elle reçoit des requêtes HTTPS (GET, POST, PUT) et répond au format JSON.

Routes principales

Route	Utilisée par	Rôle
`/auth`	Les deux applications	Connexion
`/clients`	Mobile	Prospects et clients du user connecté
`/affaires`	Mobile et bureau	Création, consultation et changement d'étape
`/catalogue`	Mobile (lecture), bureau	Compagnies et produits
`/utilisateurs`	Bureau	Gestion des comptes users
`/statistiques`	Mobile et bureau	Chiffres du user, tableau de bord du cabinet

Authentification et droits

- Connexion par e-mail et mot de passe ; le mot de passe est stocké haché avec bcrypt.
- Après connexion, l'API délivre un jeton JWT à durée limitée, transmis avec chaque requête.
- À chaque requête, l'API vérifie le jeton, puis le rôle (les routes `/utilisateurs` et la modification du catalogue sont réservées à l'admin), puis la propriété des données (un user n'accède qu'à ses propres clients et affaires).

Règles du tunnel appliquées par l'API

- Passage à « devis envoyé » : produit et prime mensuelle obligatoires.
- Passage à « signé » : numéro de contrat et date de signature obligatoires.
- Passage à « perdu » : motif obligatoire ; possible depuis n'importe quelle étape.
- Une affaire signée ou perdue ne peut plus changer d'étape.
- Chaque changement d'étape est enregistré par l'API dans `historique_etape`, dans la même transaction que la mise à jour de l'affaire.

Base de données

Une seule base MySQL, `gestion_courtage`, de 6 tables, partagée par les deux applications. Le MCD, le MLD et le schéma relationnel sont fournis à part.

Table	Contenu
utilisateur	Comptes admin et users
client	Prospects et clients, rattachés à un user
compagnie	Compagnies d'assurance partenaires
produit	Produits d'assurance et taux de commission
affaire	Ventes en cours ou terminées, avec leur étape
historique_etape	Changements d'étape (qui, quand)

Règles de conception

- Clés étrangères sur toutes les relations : une affaire ne peut pas exister sans client.
- Montants en `DECIMAL`, jamais en `FLOAT`, pour éviter les erreurs d'arrondi.
- Les étapes sont une liste fermée (`ENUM`) : prospect, qualifie, devis_envoye, signe, perdu.
- La commission n'est pas stockée : elle se calcule (prime mensuelle × taux de commission).
- Requêtes préparées exclusivement, contre les injections SQL.
- L'API utilise un compte MySQL qui peut seulement lire et modifier les données (SELECT, INSERT, UPDATE, DELETE), sans pouvoir créer ni supprimer de table.

Travaux

- [x] ~~MCD, MLD et schéma relationnel.~~
- [x] ~~Script SQL de création de la base (schema_bdd_courtage.sql, version v6).~~
- [ ] Jeu de données fictives pour les tests et la démonstration.

Sécurité et RGPD

Risque	Mesure
Vol des mots de passe	Hachage avec bcrypt, aucun stockage en clair
Interception des échanges	HTTPS entre les applications et l'API
Accès aux données d'un autre user	Contrôle du rôle et de la propriété des données à chaque requête
Injection SQL	Requêtes préparées exclusivement

Risque	Mesure
Données invalides	Validation de chaque entrée côté serveur
Fuite des secrets (mot de passe MySQL, clé JWT)	Variables d'environnement, jamais dans le code ni sur GitHub

RGPD : seules les données utiles au suivi des ventes sont collectées (identité, coordonnées, besoin) ; le développement et la démonstration utilisent uniquement des données fictives.

Installation et démonstration

- Un fichier Docker Compose lance deux conteneurs : MySQL (avec le script de création de la base) et l'API Node.js.
- Les paramètres (mots de passe, clé JWT, adresse de l'API) sont dans un fichier d'environnement non versionné.
- Démonstration : serveur sur le poste de développement, application bureau sous Windows, application mobile sur un téléphone Android ou un émulateur, avec un jeu de données fictives.

Outils et tests

- VS Code pour l'API et Flutter ; Android Studio pour l'émulateur Android.
- Postman ou Bruno pour tester l'API indépendamment des applications.
- Trello pour le planning et le suivi des tâches.
- Tests de l'API : connexion, droits (cas autorisés et refusés), règles du tunnel, calcul des statistiques.
- Cahier de recette déroulé avec le client sur les principaux scénarios.

Évolutions techniques possibles

Hors version 1 : stockage des pièces justificatives (stockage objet S3), notifications push (Firebase Cloud Messaging), e-mails automatiques (service d'e-mails transactionnels), versions iOS et macOS, hébergement en ligne et installation pour plusieurs cabinets.